

Week 4, Part 3: Widgets, Their Types and Parameters

Dr M AOUDÉ
Lecture notes

Objectives. This is the third part of week 4. By the end you will be able to: tell *visible* widgets from *layout* (invisible) widgets; list the main parameters of the widgets you use every day (Text, buttons, Image, Icon, Row, Column, Center, Padding, Stack, Scaffold); choose the right value for a parameter such as `mainAxisAlignment` or `fit`; tell the Material and Cupertino styles apart; use Scaffold (with a drawer), a custom AppBar, Expanded (with a ListView) and Container (with a BoxDecoration) correctly; and find your way in a complete program that uses all of them.

The **code** of this part is not printed here: it is in the folder `code/week04/examples` (section 7). It holds the complete program `01_widgets_full_program`, which uses all the widgets of this document, and recipes on the Scaffold (with a drawer, and in the Cupertino style), a custom AppBar, Expanded with ListView, and the Container (section 6). Part 2 (*Flutter architecture and widget types*) introduced the widget families; here we look at each widget and at **its parameters**. To run the full program, from the folder `examples`:

```
flutter run -t lib/01_widgets_full_program.dart
```

1 Widgets and parameters

A **widget** is the basic building block of every Flutter user interface. Everything is a widget: a piece of text, a button, a layout, even the whole screen. Widgets are nested inside each other, and this nesting is the widget tree.

A widget is a Dart class, and you *configure* it with the **parameters** of its constructor. Almost all of them are *named parameters* (week 2), so you write the name, then a colon, then the value:

```
1 Text(
2   'Hello, Flutter!',           // positional parameter: the text
3   textAlign: TextAlign.center, // named parameter
4   style: TextStyle(fontWeight: FontWeight.bold),
5 )
```

Three kinds of values appear again and again:

- **Simple values:** numbers (`width: 120`), strings, booleans. Sizes are in *logical pixels*.
- **Constants of an enum:** `TextAlign.center`, `MainAxisAlignment.spaceEvenly`, `BoxFit.contain`. The type name tells you which parameter accepts them.
- **Other widgets or objects:** `child: Text(...)`, `style: TextStyle(...)`, `padding: EdgeInsets.all(8)`, and **functions:** `onPressed: () { ...}`.

child or children

The first thing to know about a widget is how many widgets it can contain:

Parameter	The widget holds	Examples
child	<i>one</i> widget	<code>Center</code> , <code>Padding</code> , <code>Container</code> , <code>SizedBox</code> , <code>Card</code> , <code>Expanded</code> , <code>GestureDetector</code> , the buttons
children	a <i>list</i> of widgets, [...]	<code>Row</code> , <code>Column</code> , <code>Stack</code>

How to find the parameters of a widget. In VS Code, put the cursor on the widget and press `Ctrl+Space` inside its parentheses: the list shows every parameter with its type. `Ctrl+click` on the name opens the class, and the official reference is at api.flutter.dev. A parameter marked **required** must be given; the others are optional.

2 The two types of widgets

Widgets fall into two families. The example of this part is organised in the same way.

Type	Role	Examples
Visible widgets	show content or take input from the user; something appears on the screen	<code>Text</code> , buttons, <code>Image</code> , <code>Icon</code>
Invisible (layout) widgets	draw nothing themselves; they place and size the widgets they contain	<code>Row</code> , <code>Column</code> , <code>Center</code> , <code>Padding</code> , <code>Stack</code> , <code>Scaffold</code>

A useful picture is a page of a magazine: the text, the photos and the icons are the visible widgets; the invisible grid that decides where each of them goes is made of layout widgets. A real screen mixes both: a `Column` (invisible) holds a `Text` and a button (visible).

The word “invisible” means that the widget has no picture of its own. Some layout widgets do draw something: a `Container` with a `color` draws a rectangle, and a `Scaffold` paints the background of the page. The important point is their *role*: they organise other widgets.

3 Visible widgets and their parameters

Text

`Text` shows a string. Its first parameter is the string; the other parameters are named.

Parameter	Type	Meaning
data (first)	<code>String</code>	the text to show
style	<code>TextStyle</code>	font size, weight, color, ... (see below)
textAlign	<code>TextAlign</code>	left , right , center , justify , start , end : alignment of the lines inside the text's own width
maxLines	<code>int</code>	the maximum number of lines
overflow	<code>TextOverflow</code>	what to do with text that does not fit: clip , ellipsis , fade , visible
softWrap	<code>bool</code>	false keeps the text on one line

`style` takes a `TextStyle`, an object with its own parameters:

Parameter	Meaning	Example value
fontSize	size of the letters, in logical pixels	28
fontWeight	thickness: <code>normal</code> , <code>bold</code> , or <code>w100</code> to <code>w900</code>	<code>FontWeight.bold</code>
fontStyle	<code>normal</code> or <code>italic</code>	<code>FontStyle.italic</code>
color	color of the letters	<code>Colors.teal</code>
letterSpacing	extra space between letters	1.5
decoration	a line over, under or through the text	<code>TextDecoration.underline</code>

```

1 Text(
2   'Flutter Widgets',
3   textAlign: TextAlign.center,
4   style: TextStyle(fontSize: 28, fontWeight: FontWeight.bold, color: Colors.teal
5   ),
6 )

```

Why `textAlign` sometimes does nothing. `textAlign` aligns the lines *inside the width of the Text*. A `Text` in a `Column` whose `crossAxisAlignment` is `start` is only as wide as its letters, so there is nothing to align, and the text stays on the left. To see the effect, give the text the full width (put it in a `SizedBox(width: double.infinity, ...)`), or center the widget itself with `Center`. The title of the example of section 7 is in this case.

Buttons

A button runs a function when the user taps it. There are three main buttons; they take the same parameters and differ by their look.

Widget	Look
<code>ElevatedButton</code>	filled, with a small shadow: the main action of a screen
<code>TextButton</code>	only text, no frame: a secondary or discreet action
<code>OutlinedButton</code>	text with a thin frame: an alternative action

Parameter	Type	Meaning
child	Widget	the label, usually a <code>Text</code>
onPressed	function	runs when the button is tapped. If it is <code>null</code> , the button is <i>disabled</i> (grey)
onLongPress	function	runs when the button is pressed for a long time
style	<code>ButtonStyle</code>	colors, shadow, padding, shape; built with <code>styleFrom</code> (below)

The look of a button is set with `styleFrom`. Its main parameters are `backgroundColor`, `foregroundColor` (the color of the label), `elevation` (the depth of the shadow) and `padding`:

```

1 ElevatedButton(
2   onPressed: () => print('tapped'),
3   style: ElevatedButton.styleFrom(
4     backgroundColor: Colors.deepPurple,
5     foregroundColor: Colors.white,
6     elevation: 8,
7   ),
8   child: const Text('Elevated'),
9 )

```

Old tutorials use `RaisedButton`, `FlatButton` and `OutlineButton`, and the parameters `primary` and `onPrimary`. They were replaced by `ElevatedButton`, `TextButton`, `OutlinedButton`, `backgroundColor` and `foregroundColor`, and no longer work in recent versions of Flutter.

Image

There is one constructor for each source of the picture:

Constructor	Loads the image from
<code>Image.asset('assets/logo.png')</code>	a file of your project (an <i>asset</i>)
<code>Image.network('https://...')</code>	a URL on the Internet
<code>Image.file(file)</code>	a file on the device
<code>Image.memory(bytes)</code>	bytes already in memory

They share the same named parameters:

Parameter	Type	Meaning
<code>width</code> , <code>height</code>	<code>double</code>	the size of the box in which the image is drawn. If you give only one, the image keeps its proportions
<code>fit</code>	<code>BoxFit</code>	how the image fills the box (table below)
<code>color</code>	<code>Color</code>	a color mixed with the image (a tint)
<code>alignment</code>	<code>Alignment</code>	which part of the image stays visible when it is cropped

Value of fit	Effect
<code>BoxFit.contain</code>	the whole image is visible, proportions kept; empty space may remain
<code>BoxFit.cover</code>	the box is entirely filled, proportions kept; the image may be cropped
<code>BoxFit.fill</code>	the box is filled and the image is stretched (proportions lost)
<code>BoxFit.fitWidth</code> , <code>BoxFit.fitHeight</code>	the width (or the height) of the image matches the box
<code>BoxFit.scaleDown</code>	like <code>contain</code> , but the image is never enlarged (what you get when you give no <code>fit</code>)

To use an asset, put the file in a folder of the project (here `assets`) and declare the folder in `pubspec.yaml`, with the same indentation as below. Then run `flutter pub get`, or restart the app.

```
flutter:
  assets:
    - assets/
```

Icon

`Icon` shows one of the built-in icons of the `Icons` class.

Parameter	Type	Meaning
icon (first)	IconData	which icon: <code>Icons.favorite</code> , <code>Icons.home</code> , <code>Icons.settings</code> , ...
size	double	the width and the height of the icon (24 by default)
color	Color	the color of the icon

```
1 Icon(Icons.favorite, color: Colors.red, size: 40)
```

4 Invisible (layout) widgets and their parameters

Row and Column

`Row` places its children horizontally, `Column` places them vertically. Each has a **main axis**, the direction in which it places the children (horizontal for a `Row`, vertical for a `Column`), and a **cross axis**, the other direction.

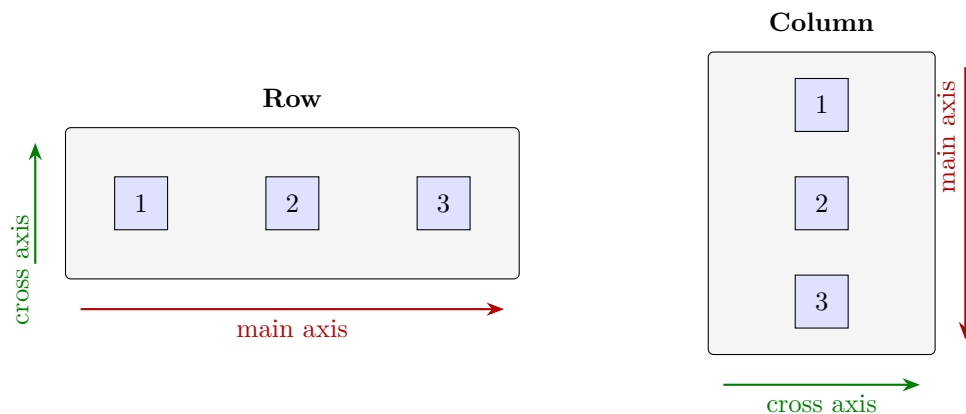


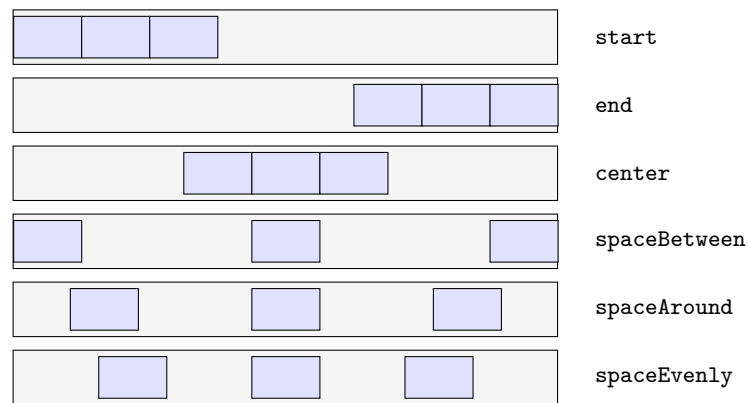
Figure 1: The two axes of a `Row` and of a `Column`.

Parameter	Type	Meaning
children	List<Widget>	the widgets to place, in order
mainAxisAlignment	MainAxisAlignment	how the free space is used <i>along</i> the main axis
crossAxisAlignment	CrossAxisAlignment	how the children are placed <i>across</i> it: start , center (the default), end , stretch
mainAxisSize	MainAxisSize	max : take all the room (the default); min : only what the children need

The six values of `mainAxisAlignment` are shown below for a `Row` of three children:

`spaceBetween` puts the free space between the children only; `spaceAround` gives each child the same space on both sides (so the space between two children is twice the space at the ends); `spaceEvenly` makes all the gaps equal, ends included. In a `Column` the same values work vertically, but the `Column` needs a height larger than its content to have free space to share.

```
1 Row(
2   mainAxisAlignment: MainAxisAlignment.spaceEvenly,
3   children: [Text('Row 1'), Text('Row 2'), Text('Row 3')],
4 )
```

Figure 2: The values of `MainAxisAlignment` in a Row.

Center and Padding

`Center` has one parameter, `child`, which it puts in the middle of the space it is given. `Padding` adds empty space around its `child`; its `padding` parameter is required and takes an `EdgeInsets`:

Constructor	Space added
<code>EdgeInsets.all(8)</code>	8 logical pixels on the four sides
<code>EdgeInsets.symmetric(horizontal: 16, vertical: 8)</code>	16 left and right, 8 above and below
<code>EdgeInsets.only(left: 4, top: 12)</code>	only on the sides you name
<code>EdgeInsets.fromLTRB(1, 2, 3, 4)</code>	left, top, right, bottom

Stack

`Stack` draws its children on top of each other: the first child is at the bottom, the last one is on top. The `alignment` parameter says where the children that are smaller than the `Stack` are placed (for example `Alignment.center`, `Alignment.topLeft`). To put one child at an exact place, wrap it in a `Positioned` widget with `top`, `left`, `right`, `bottom` (this is what example 14_gestures does).

Scaffold and AppBar

`Scaffold` gives a page its standard structure. Each parameter is a *slot* that takes a widget:

Parameter of Scaffold	Slot
<code>appBar</code>	the bar at the top, usually an <code>AppBar</code>
<code>body</code>	the content of the page
<code>floatingActionButton</code>	the round button floating over the content
<code>bottomNavigationBar</code>	a bar at the bottom
<code>drawer</code>	the side menu
<code>backgroundColor</code>	the color of the page

`AppBar` has its own parameters: `title` (a widget, usually a `Text`), `centerTitle`, `backgroundColor`, `foregroundColor` (the color of the title and the icons), `leading`, `actions` (a list of widgets on the right) and `elevation` (the shadow below the bar).

Widgets that help the layout

These are small layout widgets used in almost every screen:

Widget and parameters	Role
<code>SizedBox(width, height, child)</code>	a box of a fixed size; with no child it is an empty space between two widgets
<code>Container(child, color, width, height, padding, margin, alignment, decoration)</code>	a rectangle that can have a size, a color, a border, space inside (padding) and outside (margin); see recipe 5
<code>Card(child, elevation, color, margin)</code>	a rectangle with rounded corners and a shadow
<code>Expanded(flex, child)</code>	inside a <code>Row</code> or a <code>Column</code> : takes the free space; flex is the share (example <code>12_expanded</code>)
<code>SingleChildScrollView(child, padding)</code>	makes a page that is too long scrollable

A `Container` cannot have both `color` and `decoration`: `color` is only a shortcut for a decoration with a background color. Use one or the other.

5 The common parameters at a glance

The same parameter names come back in many widgets. Learning them once is worth more than learning each widget separately:

Parameter	Meaning	Used by
<code>child</code>	the one widget inside	<code>Center</code> , <code>Padding</code> , <code>Container</code> , <code>buttons</code> , <code>Card</code>
<code>children</code>	the list of widgets inside	<code>Row</code> , <code>Column</code> , <code>Stack</code>
<code>padding</code>	space <i>inside</i> a widget, around its child	<code>Padding</code> , <code>Container</code> , <code>SingleChildScrollView</code>
<code>color</code>	background or foreground color	<code>Container</code> , <code>Icon</code> , <code>Card</code> , <code>TextStyle</code>
<code>width, height</code>	size of the widget	<code>Container</code> , <code>SizedBox</code> , <code>Image</code>
<code>alignment</code>	where the child (or children) is placed inside the widget	<code>Stack</code> , <code>Container</code>
<code>mainAxisAlignment</code>	how children are spread along the main axis	<code>Row</code> , <code>Column</code>
<code>crossAxisAlignment</code>	how children are placed across the main axis	<code>Row</code> , <code>Column</code>
<code>onPressed</code>	function run when the button is tapped	<code>buttons</code> , <code>IconButton</code>
<code>style</code>	the look: a <code>TextStyle</code> or a <code>ButtonStyle</code>	<code>Text</code> , <code>buttons</code>
<code>elevation</code>	depth of the shadow	<code>buttons</code> , <code>AppBar</code> , <code>Card</code>
<code>textAlign</code>	alignment of the lines of a text	<code>Text</code>

6 Recipes: Scaffold, AppBar, Expanded and Container

The recipes below use the widgets of sections 3 to 5 to build the structure of a real screen. For each one we give the problem, the solution, and the points to remember. The code is in the `examples` project (section 7); it is not repeated here.

Two design languages: Material and Cupertino

Android and iOS user interfaces are based on two different design languages. Android uses **Material Design**; Apple has the **Human Interface Guidelines**. The Flutter team calls the iOS style **Cupertino**, after the city where Apple has its headquarters. Two libraries provide the ready-made widgets of the two styles: `material.dart` and `cupertino.dart`.

Both libraries have a widget that gives a screen its basic structure, and both work on *every* platform, because Flutter draws the widgets itself: you choose a style, not an operating system.

	Material	Cupertino
The application	<code>MaterialApp</code>	<code>CupertinoApp</code>
The structure of a screen	<code>Scaffold</code>	<code>CupertinoPageScaffold</code>
The bar at the top	<code>AppBar (title)</code>	<code>CupertinoNavigationBar (middle)</code>
The content	<code>body</code>	<code>child</code>
The button	<code>ElevatedButton</code>	<code>CupertinoButton</code>
Bottom tabs	<code>BottomNavigationBar</code>	<code>CupertinoTabScaffold + CupertinoTabBar</code>
Side drawer	<code>Drawer</code>	(none: iOS has no drawer)

A name to avoid. Some texts speak of a `CupertinoScaffold`. There is no widget with this name. In the Cupertino library the structure of *one screen* is a `CupertinoPageScaffold`, and the structure of an app with tabs is a `CupertinoTabScaffold`. Example 07_cupertino_scaffold shows the first one.

Recipe 1: the Scaffold

Problem. You want the standard Material layout for a page. **Solution.** Use a `Scaffold`, with its slots `appBar`, `body`, `floatingActionButton` and `bottomNavigationBar`, as in example 02_scaffold_bottom_bar.

- A `Scaffold` **expands to fill the screen** and adapts when the screen changes (rotation, a smaller window). When the on-screen **keyboard** opens, the body becomes smaller by itself: this is the default (`resizeToAvoidBottomInset` is `true`), and you write no code for it. The test of the example checks it.
- Use **one** `Scaffold` per page, at the top, inside `MaterialApp`. Do not nest a `Scaffold` in another one: it is the top-level container of a page.
- A `BottomAppBar` has a **color** and a **shape**. With `CircularNotchedRectangle` it gets a round notch, and a `FloatingActionButton` placed with `FloatingActionButtonLocation.centerDocked` sits in the notch. Keep the text of the bar away from the middle, where the button is.
- A floating button usually *changes something on the page*, here a counter. Data that changes needs a **stateful** widget and `setState` (part 2, section on stateless and stateful widgets, and example 00_stateless_vs_stateful).

```

1 Scaffold(
2   appBar: AppBar(title: const Text('Scaffold Example')),
3   backgroundColor: Colors.blueGrey,
4   body: Center(child: card),
5   floatingActionButton: FloatingActionButton(onPressed: _addTap, child: const
6     Icon(Icons.add)),
7   floatingActionButtonLocation: FloatingActionButtonLocation.centerDocked,
8   bottomNavigationBar: const BottomAppBar(color: Colors.blueAccent, shape:
9     CircularNotchedRectangle(), child: label),
10 )

```

Recipe 2: an AppBar and a slide-out Drawer

Problem. You want to give your app some structure: a bar at the top, and a menu that you pull out from the side. **Solution.** Give the `Scaffold` a `drawer`, as in example 06_scaffold_drawer.

- The `drawer` slot takes a `Drawer`: a panel that slides in from the **left**. The `endDrawer` slot does the same from the **right**.
- You do *not* write the button that opens it. When a `Scaffold` has a drawer, its `AppBar` adds the “menu” button (three lines) by itself, and a button at the right end for an `endDrawer`. The user can also pull the drawer out with a swipe from the edge of the screen.
- A drawer usually holds a `ListView` with a `DrawerHeader` and some `ListTile` rows. Give the `ListView` padding: `EdgeInsets.zero`, so that the header reaches the top of the screen.
- A `ListTile` has `leading`, `title`, `subtitle`, `trailing`, `selected` and `onTap`, the function that runs when the row is tapped.
- The drawer **does not close by itself** when a row is tapped. It is a temporary layer above the page, and `Navigator.pop(context)` removes it. In the example, a tap on a row changes the chosen page in `setState` and closes the drawer.

```

1 drawer: Drawer(
2   child: ListView(
3     padding: EdgeInsets.zero,
4     children: [
5       const DrawerHeader(child: Text('Menu')),
6       ListTile(title: const Text('Settings'), onTap: () => Navigator.pop(context)
7     ),
8   ],
9 ),

```

To open a drawer from your *own* button, write `Scaffold.of(context).openDrawer()`. The `context` must be *below* the `Scaffold`: in the build method that creates the `Scaffold` it is above it, and Flutter reports *Scaffold.of()* called with a context that does not contain a `Scaffold`. Wrap the button in a `Builder`, or use the automatic button.

Recipe 3: an AppBar in its own class

Problem. You want to show a header at the top and keep its code in a class of its own. **Solution.** Write a class that *implements* `PreferredSizeWidget`, as in example 03_custom_appbar.

- The `appBar` slot of a `Scaffold` must know the **height** of the bar *before* drawing it. A widget that goes in this slot therefore gives its height in the getter `preferredSize`. `AppBar` does it already; your own class must do it too.

- The `build` method of your class returns a normal `AppBar`. Give the **same number** to `preferredSize` and to `toolbarHeight`, so that the title is centered in the bar that you announced.
- The main properties: `title`, `backgroundColor`, `elevation` (0 is a flat bar, more than 0 is a raised bar with a shadow), `leading` (the left end, for a menu icon) and `actions` (the right end, a list of widgets).
- With a dark `backgroundColor`, also set `foregroundColor`: the default color of the title and of the icons may be dark too, and invisible on black.

```

1 class MyAppBar extends StatelessWidget implements PreferredSizeWidget {
2   const MyAppBar({super.key, required this.title});
3   final String title;
4
5   @override
6   Size get preferredSize => const Size.fromHeight(100);
7
8   @override
9   Widget build(BuildContext context) => AppBar(title: Text(title), toolbarHeight
10    : 100);
11 }

```

Two styles of constructor. Many books and tutorials write `const MyApp({Key? key}) : super(key: key);`. Since Dart 2.17 the short form `const MyApp({super.key});` does the same and is what the examples of this course use. Both are correct; you will see both in code that you read.

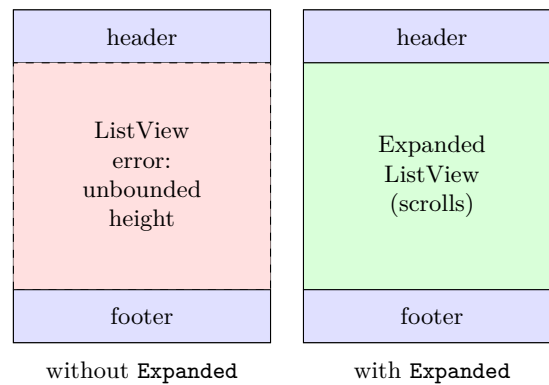
Recipe 4: Expanded

Problem. You want to use all the space that is available on the screen, without asking the screen for its size. **Solution.** Put the widgets in a `Column` or a `Row` and wrap the ones that should grow in `Expanded` (examples `04_expanded_flag` and `05_expanded_listview`; the `flex` parameter is in example `12_expanded` of the first project).

- An `Expanded` child fills the space that the other children *do not use*. A child that is not `Expanded`, like a title, keeps its own size.
- If you do not write `flex`, it is 1: several `Expanded` children share the free space **equally**. In example `04_expanded_flag`, four areas around one line of text have exactly the same height, on any screen.
- An `Expanded` with an empty `Container` is a flexible empty space; a `Container` with a color is a colored area.
- `RichText` with a `TextSpan` shows a text whose parts can have different styles (the example has only one span). For a simple text, `Text` is enough.

The ListView.

`Expanded` is most useful with lists. A `ListView` is a scrolling list; `ListTile` is a ready-made row (`leading`, `title`, `subtitle`, `trailing`); `ListView.builder` builds the rows only when they are about to appear, so it works for long lists. By itself a `ListView` tries to take *all* the height that it can get. Inside a `Column` it is given an unlimited height, and Flutter stops with an error. Wrapping it in `Expanded` tells it to take *exactly the remaining height* of the screen, the part that the other children of the `Column` do not use, and to scroll inside it. The header and the footer stay where they are.



Recipe 5: the Container and its visual effects

Problem. You want to draw a box: with a size, space around and inside, a color, rounded corners, a shadow. **Solution.** Use a **Container** and give it a **decoration** (example `08_container_effects`). **Container** is one of the most versatile widgets: it combines several small widgets (**Padding**, **Align**, **ConstrainedBox**, **DecoratedBox**, **Transform**) in one.

Parameter of Container	Meaning
width, height	the size of the rectangle
margin	empty space <i>outside</i> the rectangle
padding	empty space <i>inside</i> it, between its edge and the child
alignment	where the child is placed in the rectangle (<code>Alignment.center</code> , <code>Alignment.bottomRight</code> , ...)
decoration	how the rectangle is drawn: a BoxDecoration (below)
transform	a <code>Matrix4</code> that rotates, moves or scales the box, for example <code>Matrix4.rotationZ(-0.1)</code> (an angle in radians)
child	the widget inside

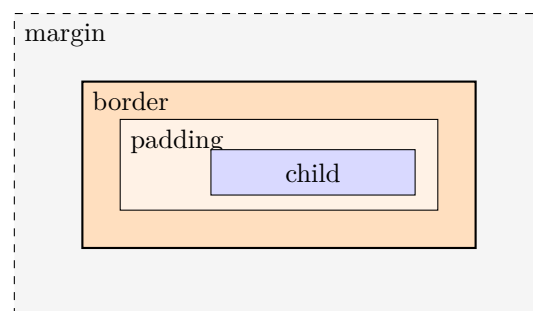


Figure 3: The layers of a **Container**, from the outside: the margin, the decorated rectangle (with its border), the padding and the child.

The **BoxDecoration** has its own parameters:

Parameter	Meaning	Example value
color	the background color	Colors.white
border	a line around the box	Border.all(color: Colors.blue, width: 3)
borderRadius	rounded corners	BorderRadius.circular(16)
boxShadow	a list of shadows; a BoxShadow has color, blurRadius and offset	[BoxShadow(color: Colors.black38, blurRadius: 8, offset: Offset(4, 4))]
gradient	a blend of colors instead of one color	LinearGradient(colors: [Colors.orange, Colors.purple])
shape	BoxShape.rectangle (the default) or BoxShape.circle	BoxShape.circle

```

1 Container(
2   margin: const EdgeInsets.all(16),
3   padding: const EdgeInsets.all(16),
4   decoration: BoxDecoration(
5     color: Colors.white,
6     border: Border.all(color: Colors.blue, width: 3),
7     borderRadius: BorderRadius.circular(16),
8   ),
9   child: const Text('A card made with a Container'),
10 )

```

- `Container(color: c)` is only a shortcut for `Container(decoration: BoxDecoration(color: c))`. You **cannot give both** color and decoration: Flutter stops with *Cannot provide both a color and a decoration*. When you use a decoration, put the color in it.
- The `Container` itself includes its `margin`: the rectangle that you see is smaller than the widget by the margin on each side.
- For a circle, give the same `width` and `height` and `shape: BoxShape.circle`; do not combine `shape: BoxShape.circle` with a `borderRadius`.
- With no `child`, no `width` and no `height`, a `Container` takes all the space it is given; with a `child`, it takes the size of its child.

Common errors

Message or symptom	Cause	Fix
<i>Vertical viewport was given unbounded height</i>	a <code>ListView</code> (or other scrolling widget) directly in a <code>Column</code>	wrap it in <code>Expanded</code>
Title and icons of the <code>AppBar</code> are invisible	dark <code>backgroundColor</code> and dark default text color	set <code>foregroundColor</code>
A custom bar is refused by the <code>appBar</code> slot	the class does not implement <code>PreferredSizeWidget</code>	add <code>implements PreferredSizeWidget</code> and <code>preferredSize</code>
The floating button does not change the screen	the page is stateless, or <code>setState</code> is not called	make the page stateful; change the data in <code>setState</code>
Text in the bar is hidden behind the floating button	the docked button covers the middle of the <code>BottomAppBar</code>	put the text on the left or on the right
The drawer stays open after a tap on a row	nothing removes the layer	call <code>Navigator.pop(context)</code> in <code>onTap</code>
<i><code>Scaffold.of()</code> called with a context that does not contain a <code>Scaffold</code></i>	the <code>context</code> is above the <code>Scaffold</code> (the one of the build method that creates it)	use a <code>Builder</code> , or the automatic menu button
<i>Cannot provide both a color and a decoration</i>	<code>color</code> and <code>decoration</code> in the same <code>Container</code>	put the color in the <code>BoxDecoration</code>

7 The programs

The code of this part is **not** printed in this document. It is in the folder `code/week04/examples`, a small Flutter project with one runnable app per file. Each file starts with a comment that tells you what you learn, how to run it and what you should see, and the file `test/widget_test.dart` checks it automatically. See the guide *Running the Flutter apps in VS Code* to prepare the tools; then, from the folder **examples**:

```
flutter pub get
flutter test
flutter create . (one time only)
flutter run -t lib/01_widgets_full_program.dart
```

File	What it shows	Notes
01_widgets_full_program	the full program: visible widgets, layout widgets and their common parameters in one page; the last tapped button is remembered	sections 3 to 5
02_scaffold_bottom_bar	Scaffold, BottomAppBar with a notch, docked floating button, counter, text field and keyboard	recipe 1
03_custom_appbar	an AppBar in its own class: PreferredSizeWidget, leading, actions, elevation	recipe 3
04_expanded_flag	four Expanded children share the height equally; RichText	recipe 4
05_expanded_listview	a ListView in an Expanded, with a header and a footer	recipe 4
06_scaffold_drawer	drawer and endDrawer, DrawerHeader, ListTile, Navigator.pop	recipe 2
07_cupertino_scaffold	CupertinoApp, CupertinoPageScaffold, CupertinoNavigationBar, CupertinoButton	Material and Cupertino
08_container_effects	Container: margin, padding, BoxDecoration, gradient, circle, alignment, transform	recipe 5

The full program, part by part

The file `01_widgets_full_program.dart` has three parts, like sections 3 to 5: visible widgets, invisible widgets, and the common parameters. The page is a `StatefulWidget`, because it remembers which button was tapped last; everything else is stateless. The structure of the widget tree is:

WidgetsFullProgramApp	(StatelessWidget)
MaterialApp	
HomePage	(StatefulWidget, its State keeps <code>_lastButton</code>)
Scaffold	
AppBar	title, backgroundColor, foregroundColor
SingleChildScrollView	padding
Column	crossAxisAlignment: start
Text	the title, then an introduction
1) Visible widgets:	Center > Column > Text, Row of 3 buttons, Image, Icon, Text (the last button)
2) Layout widgets:	Row, Column (in a Container), Center, Stack
3) Parameters:	Text and a Card

Open the file and look at these lines:

Lines	What to look at
33--52	main starts the app. <code>MaterialApp</code> sets the title and a theme built from one <i>seed color</i> ; its home is the page

54--68	the stateful page: one piece of data that changes, <code>_lastButton</code> . A button calls <code>_selectButton</code> , which changes it inside <code>setState</code>
70--98	the Scaffold with its appBar; the body is a <code>SingleChildScrollView</code> (a long page can scroll) with a <code>padding</code> and a <code>Column</code> ; the first texts use <code>style</code> and <code>textAlign</code>
100--164	part 1, the visible widgets: a <code>Text</code> , a <code>Row</code> of three buttons (<code>onPressed</code> calls <code>_selectButton</code> ; the <code>ElevatedButton</code> also has a <code>style</code>), an <code>Image.asset</code> with <code>width</code> , <code>height</code> and <code>fit</code> , an <code>Icon</code> with <code>color</code> and <code>size</code> , and a <code>Text</code> that shows <code>_lastButton</code>
166--216	part 2: a <code>Row</code> with <code>spaceEvenly</code> and three <code>Padding</code> ; a <code>Column</code> in a <code>Container</code> with a <code>color</code> and a <code>padding</code>
218--269	part 2: a <code>Container</code> with <code>width</code> and <code>height</code> inside a <code>Center</code> ; a <code>Stack</code> of a blue square, a smaller yellow square and a text, all centered by <code>alignment</code>
271--322	part 3: the list of the parameters in one <code>Text</code> (a long string cut in several lines, joined by <code>Dart</code>) and three examples of code in a <code>Card</code>

What you should see

A scrollable page with a teal `AppBar`. In part 1: the text *Hello, Flutter!*, three buttons, the logo, a red heart and the line *No button selected*; tapping a button changes this line to *ElevatedButton*, *TextButton* or *OutlinedButton*. In part 2: a row of three labels, three lines on an amber background, an orange box with a centered label, and the stack of two squares with the word *Stack*. In part 3: the list of parameters and a grey card with three lines of code.

8 Exercises

1. Change `mainAxisAlignment` of the `Row` of labels to `spaceBetween`, then to `end`. Explain what moves.
2. Make the title *Flutter Widgets* really centered: first with a `SizedBox(width: double.infinity, ...)`, then with a `Center`. Which `textAlign` is still needed?
3. Change the `fit` of the image to `BoxFit.fill` and give it a `width` of 250 and a `height` of 100. What happens to the logo? Try `BoxFit.cover`.
4. Set `onPressed: null` on the `TextButton`. What does the button look like?
5. Add a fourth child to the `Stack`: an `Icon` inside a `Positioned(top: 8, right: 8, ...)`.
6. Add `elevation: 12` in the `styleFrom` of the `ElevatedButton`, and `elevation: 0` in the `AppBar`. Where do you see the difference?
7. In example `02_scaffold_bottom_bar`, remove `floatingActionButtonLocation`: where does the button go? Then change the `shape` of the `BottomAppBar`.
8. In example `03_custom_appbar`, remove `foregroundColor`: what happens to the title? Then change the `height` in `preferredSize` only, and not in `toolbarHeight`: what do you see?
9. In example `04_expanded_flag`, give the first `Expanded` `flex: 2`. How do the heights change?
10. In example `06_scaffold_drawer`, remove `Navigator.pop(context)`: what happens after a tap on a row? Add a fourth row.
11. In example `07_cupertino_scaffold`, replace `CupertinoPageScaffold` by a `Scaffold`: what must you change (the app, the bar, the button)?
12. In example `08_container_effects`, change `borderRadius` of box 2 to `BorderRadius.circular(40)`, then move the shadow with `offset: Offset(-4, 4)`. Add `color: Colors.red` to box 2

next to its `decoration` and read the error.

13. In example 05_expanded_listview, remove the `Expanded` around the `ListView` and read the error message. Which words tell you the cause?

9 Summary

Idea	In one sentence
Widget	the basic block of a Flutter interface; you configure it with the parameters of its constructor
Visible widgets	show content or take input: <code>Text</code> (<code>style</code> , <code>textAlign</code>), <code>buttons</code> (<code>child</code> , <code>onPressed</code> , <code>style</code>), <code>Image</code> (<code>width</code> , <code>height</code> , <code>fit</code>), <code>Icon</code> (<code>size</code> , <code>color</code>)
Invisible widgets	organise other widgets: <code>Row</code> and <code>Column</code> (<code>children</code> , <code>mainAxisAlignment</code> , <code>crossAxisAlignment</code>), <code>Center</code> , <code>Padding</code> (<code>padding</code>), <code>Stack</code> (<code>alignment</code>), <code>Scaffold</code> (<code>appBar</code> , <code>body</code>)
child / children	one widget inside / a list of widgets inside
Main and cross axis	the direction of a <code>Row</code> or <code>Column</code> , and the other one
Material / Cupertino	two design languages, both usable on every platform: <code>MaterialApp</code> + <code>Scaffold</code> + <code>AppBar</code> , or <code>CupertinoApp</code> + <code>CupertinoPageScaffold</code> + <code>CupertinoNavigationBar</code>
Drawer	the <code>drawer</code> / <code>endDrawer</code> of a <code>Scaffold</code> ; the menu button is automatic; close it with <code>Navigators.pop</code>
Container	<code>width</code> , <code>height</code> , <code>margin</code> (outside), <code>padding</code> (inside), <code>alignment</code> , <code>transform</code> , and a <code>BoxDecoration</code> (<code>color</code> , <code>border</code> , <code>borderRadius</code> , <code>boxShadow</code> , <code>gradient</code> , <code>shape</code>); never <code>color</code> and <code>decoration</code> together
Scaffold	one per page, at the top: fills the screen, follows the keyboard; slots <code>appBar</code> , <code>body</code> , <code>bottomNavigationBar</code> , <code>floatingActionButton</code>
Custom AppBar	implements <code>PreferredSizeWidget</code> and gives its height in <code>preferredSize</code>
Expanded	fills the free space of a <code>Column</code> or <code>Row</code> ; equal shares without <code>flex</code> ; needed around a <code>ListView</code> in a <code>Column</code>
Stateless / stateful	fixed content / content that changes; <code>setState</code> rebuilds a stateful widget

This document is maintained by Dr. Mohamad AOUE

Last updated: 2026-09-21